

UNIVERSITY OF LIVERPOOL

Project 6: Temporal Path Explorer & Visualizer

Final Report

By Team 9:

Ankit, Ranjan

Liu, Yiqing

Wang, Ce

Zeng, Wenbin

Li, Bowen

Tang, Zixuan

Xu, Xiaohan

Supervisor: Lutz Oettershagen

Module: COMP530

Submission Date: 8 May 2026

Source Code Link: https://github.com/feinal/comp530_team9

1. Aims, Objectives, and Milestones

1.1 Aims

This project, the Temporal Path Explorer & Visualizer, was built to give users a way to study reachability in temporal networks through a working web application. Most graph tools we reviewed during planning are made for static graphs and do not show what happens when edges only exist at certain times, or when a path is only valid if its edges occur in non-decreasing time order. We wanted to close that gap. Putting a temporal reachability calculation behind a visual interface makes the idea of a time-respecting path much easier to see. The work is implementation-focused. A user should be able to upload data or generate a small dataset, pick a starting node, and watch the set of reachable nodes grow as the clock moves forward. They should also be able to ask what-if questions, such as what happens if an edge is removed, and see the answer straight away. This kind of exploration was missing from the tools we found, since most existing visualisers ignore time completely.

We also wanted the tool to be useful for both analysis and teaching. The core search runs in a single pass, so it scales reasonably, and the front end uses Streamlit to keep the output light. The what-if feature was the part we found most interesting, because temporal networks are very sensitive to small changes. A single removed edge can cut off a whole branch downstream, and we wanted users to see this for themselves.

1.2 Definition

Temporal graph. In this report, a temporal graph is modelled as an edge list where each directed interaction connects a source and a target and carries a numerical timestamp t (stored as columns SRC, TGT, Unix). This is a discrete-time, labelled-edge representation: edges are instantaneous events occurring at discrete times, consistent with Holme and Saramaki (2012) [1].

Time-respecting path (temporal walk). A feasible route from a start node s to a node x traverses outgoing edges $e_1, \dots, e_k$ whose timestamps form a non-decreasing sequence. "Non-decreasing" means equal timestamps across consecutive hops are permitted; timestamps must not move backwards in time.

Earliest-arrival path. Among all time-respecting routes from s (subject to optional departure constraints), we focus on minimising arrival time at a target rather than unweighted shortest path in hop count.

Global reachability and mean latency (project metrics). We report two aggregate indicators computed on the cleaned graph structure: (i) global reachability as an overall temporal connectivity statistic, and (ii) mean latency summarising temporal delay across reachable destinations under the adopted metric computation (baseline used consistently across tabs).

Cumulative interaction view (UI). The main plot uses a time cutoff t to display the subgraph of edges with $\text{Unix} \leq t$ (cumulative view) with a fixed layout; this filter controls what is drawn, while path queries can typically use the full uploaded table unless stated otherwise.

1.3 Objectives

The project had six objectives.

First, build a robust data layer that supports CSV/JSON input and standardises all records into SRC, TGT, and Unix fields.

Second, implement temporal reachability with non-decreasing time constraints and earliest-arrival computation over a time-ordered edge list.

Third, deliver a Streamlit-based interface for upload, parameter input, and interactive visual output.

Fourth, support time-based progression through a cumulative slider so users can observe reachability as time advances.

Fifth, provide what-if analysis through node/edge removal and compare baseline versus modified metrics.

Sixth, organise delivery through modular team development and final integration into one workflow.

1.4 Project Milestones Review

This project has successfully processed the key milestones defined in the Proposed Design report. More details can be identified as follows:

M1: Data Input Standardisation (Week 1): Cleared definition of temporal network standards and implementation of formatted data uploading and parsing.

Established the (u, v, t) data structure and parsing logic.

M2: Algorithm Verification (Week 4): Successfully processed implementation and verification of the temporal reachability BFS. Set up a static data frame for data processing.

M3: System Integration (Week 6): Developed the integration of UI and the verified temporal reachability algorithm. Achieved visualisation using the Streamlit front end.

M4: Time Control Mechanism (Week 6): Developed a time-threshold-based cumulative slider to view the visible edge subgraphs at different moments.

M5: What-If Functionality (Week 7): Implemented the what-if feature to delete selected edges or nodes.

1.5 Extent to which Aims and Objectives were Met

All primary aims and objectives were met. The system supports temporal data loading, earliest arrival path exploration, what-if deletion analysis and bottleneck scanning. In the current implementation, to ensure operational stability, bottleneck scanning will be disabled on the Advanced page when the number of edges exceeds 500.

2. Team

2.1 Team Members and Roles

Table 1: Detailed Coding Function Ownership

Team Member	Primary Role	Key Function
Yiqing LIU	System Integration	<code>build_interface</code> (UI Layout & Wiring)
Ankit RANJAN	Data Engineering	<code>standardize_data</code> (Cleaning & Sorting)
Xiaohan XU	Core Algorithms	<code>calculate_earliest_arrival</code> (Temporal BFS Algorithms)
Bowen LI	Simulation Dev	<code>apply_what_if_modifications</code> (Scenarios)
Zixuan TANG	Optimization	<code>find_menger_bottlenecks</code> (Bottleneck Scan)
Ce WANG	Layout Design	<code>compute_static_layout</code> (Coordinate Mapping)
Wenbin ZENG	Visual Interaction	<code>highlight_active_elements</code> (UI Enhancement)

2.2 Individual Role Summaries

- Yiqing LIU: Main control of the project design and management. Responsible for final integration and overall code review.
- Ankit RANJAN: Responsible for data cleaning and standardisation. Designed the input API.
- Xiaohan XU: Developed the BFS algorithm structure, which implements the earliest arrival search mechanism.
- Bowen LI: Developed and tested the what-if functionality.
- Zixuan TANG: Code optimisation, developed the bottleneck scanning function.
- Ce WANG: Designed the static layout data frame, implemented screen snapshot.
- Wenbin ZENG: Path highlighting and scene comparison visualisation.

3. Outcome

3.1 Key deliverable: Interactive temporal network analysis tool

This project has culminated in the development of a fully-featured web application dedicated to the exploration and analysis of temporal network data. Unlike traditional static graph analysis, the system emphasises the central role of the temporal dimension in the evolution of connectivity. Developed using Python's Streamlit framework, the system provides users with an intuitive interface that requires no prior knowledge of front end development.

3.2 Data processing and standardisation

The system features a robust data layer capable of handling common formats, including CSV and JSON.

Data import: Users can upload custom data or use the built-in sample dataset.

Structural requirements: The dataset must contain three core fields:

Source node (SRC): Identifier of the interaction origin.

Target node (TGT): Identifier of the interaction destination.

Unix timestamp (Unix): Numeric event time used for chronological ordering.

Pre-processing workflow: The system features built-in data cleansing capabilities, including the removal of missing values, the handling of duplicate records, and strict chronological sorting. This process ensures a clean, chronologically ordered edge list for analysis.

3.3 Algorithm implementation

The core logic of the system is based on an improved Temporal Breadth-First Search (Temporal BFS) algorithm.

Temporal path constraint: Unlike traditional BFS, this project defines a valid path as one that must follow a non-decreasing temporal order. This means that edges must be activated sequentially over time, reflecting the impact of temporal constraints on the spread of information or the transmission of epidemics.

Reachability expansion: The algorithm computes reachable nodes stepwise, reflecting the temporal expansion of influence. It determines the earliest possible arrival time at each node from a selected source, enabling the study of time-dependent propagation.

Single-pass processing: The implementation processes a time-ordered edge list in a single pass (M edges), which supports practical responsiveness on medium-sized datasets.

3.4 Dynamic Visualisation and UI Interaction

Using Plotly and NetworkX, the system transforms abstract algorithmic results into intuitive visual feedback.

Time control: Users can control the time dimension using a slider to observe how nodes and edges gradually become reachable over time.

Visual encoding: The system employs incremental rendering and caching mechanisms to dynamically highlight active paths at the current timestamp, thereby bridging the gap between theoretical models and practical understanding.

3.5 Additional features: what-if scenario analysis

This is a key feature of the project, designed to reveal the high sensitivity of temporal networks to individual events.

1. Interactive editing: Allows users to delete specific nodes or edges directly on the interface.
2. Real-time recalculation: The system immediately recalculates and displays the impact of these changes on global reachability and mean latency.
3. Menger-inspired bottleneck analysis: This feature allows users to identify temporal bottlenecks—critical edges or nodes whose removal significantly disrupts connectivity. By observing how removals of nodes or edges affect reachability, this provides an empirical illustration consistent with Menger-inspired intuition in a discrete-time environment, such as simulating the failure of a key transport hub or communication link.

3.6 Overall Contribution

The project delivers an end-to-end workflow for temporal network analysis, from data loading and cleaning to path exploration, what-if simulation, and bottleneck ranking within a single Streamlit application.

4. Implementation and Challenges

4.1 Implementation

4.1.1 System Design

The code of this system is based on Python, which utilizes modular layering to divide into three layers, including the data processing layer, the algorithm layer, and the visualisation layer. Each module is connected by a uniform interface, which realizes the decoupling of data preprocessing, graph modelling, and algorithm analysis. Finally, the system is integrated via Streamlit, which provides an interactive UI to support dynamic analysis and result presentation for users.

4.1.2 Data preprocessing

This system accepts CSV and JSON data input and uses Pandas to process the dataset. Field mapping unifies the data into three columns: SRC, TGT, and Unix. Then it cleans missing values and removes duplicate records. After that, it sorts the data by timestamp to generate standardised temporal edge data.

4.1.3 Modelling the Temporal Graph

After data preprocessing, the data is transformed into a temporal graph which is based on an adjacency list. Each edge in the graph is stored in the format of timestamp, target node and edge ID, which makes graph structure able to express topological relationships and include time information simultaneously. This representation method supports path searching under time constraints.

4.1.4 Algorithm

The graph algorithm of the system includes three parts: earliest-arrival path calculation, network sensitivity analysis, and bottleneck detection. The following will introduce them one by one.

1. The earliest-arrival path

This module is utilized to calculate the earliest-arrival path under the time constraint. The algorithm of this part is based on Breadth-First Search (BFS), and introduces a non-decreasing time constraint. During the search, it merely allows visiting the edge whose time is not earlier than the current state to ensure the time consistency of the path. When implementing this function, it uses deque to manage the queue and uses a hash table to store the earliest arrival time of each node, and maintains predecessor nodes to enable path reconstruction.

2. Network sensitivity analysis

This module is used to evaluate the influence on network structure when nodes or edges are removed. It uses a what-if method to remove elements to recalculate the reachability and average latency of the network. This process is based on a time-restricted algorithm implementation and duplicates the graph structure to avoid influence on the original data. By comparing the metrics before and after removing elements, it enables the quantification of the importance of each node and edge.

3. Bottleneck detection

This module is employed to identify the critical nodes and edges which are most influential for network performance. This method removes elements individually and calculates the variation in network metrics. Then, according to the reachability decrease and average latency increase, the results are sorted to filter the critical structures.

This method is essentially an approximation approach based on sensitivity analysis and is inspired by Menger's theorem [2]. It is used to discover potential bottleneck nodes and edges in the network.

4.1.5 Visualisation

The visualisation is based on NetworkX and Plotly. First, the system uses a spring layout to calculate the position of nodes, and on this basis constructs a stable temporal network view. The system supports time-based filtering, path highlighting, critical-node/edge marking, and before/after metric comparison with in-graph highlighting of removed elements.

4.1.6 System Integration

This system integrates each module through Streamlit, which implements an integrated process of data input, function calling, and result visualisation. Users are able to input data and select analysis functions; the system will call the corresponding module to return results according to user input.

4.2 Challenges

4.2.1 Development

In the early stage of project development, we established several core conventions through research on temporal network problems. First, we uniformly adopted DataFrame → adj as the cross-module data contract, where edge entries

are represented as (timestamp, target, edge_id) to preserve both temporal and topological information simultaneously. Second, in terms of project scope definition, we decided that what-if focuses only on removal scenarios and excludes delay operations to control complexity and ensure consistent result standards. This decision aligns with the common time-stamped edge modelling approach in temporal network research (Holme & Saramäki, 2012) [1]. At the methodological level, the team decided to advance work in parallel and retained three core algorithm directions during the subsequent integrated development: time-respecting earliest-arrival, what-if removal, and bottleneck ranking.

4.2.2 Integration

During integration, module-level code was mostly functional, but data-cleaning outputs were initially inconsistent with the agreed interface, which caused merge-stage errors.

To address this, the team added a dedicated adaptation layer (data_adj) to centralise format conversion and reduce cross-module coupling.

For the integrated interface, the final rendering stack kept NetworkX for layout and adopted Plotly for interaction and display consistency.

To protect runtime stability in the final build, the Advanced module applied a 500-edge threshold, and unfinished animation plans were not integrated.

5. Data, Analysis and Interpretation

5.1 Data and Analysis

5.1.1 Experimental Inputs

The project uses two types of data. The first is the built-in small dataset demo_data.csv (20 nodes, 48 edges, time range 0–19), used for main workflow verification and point-to-point result demonstration. The second is large-scale sx-mathoverflow-related data, which is mainly used during module development for investigating and validating the cleaning workflow, while the main workflow demonstration remains based on small-scale data.

All inputs are mapped to three canonical columns, SRC, TGT, and Unix, before entering the algorithm and metrics modules.

5.1.2 Experimental Procedure

The experimental procedure follows a development verification and web-side comparison process. First, a small-scale validation module is built and point-

to-point examples that can be calculated by hand are selected. Second, path and arrival-time results are manually computed under the same input. Third, earliest-arrival computation is run on the web interface using the same input. Finally, manual results are compared with web output for consistency.

5.1.3 Experimental Outputs

On `demo_data.csv`, baseline metrics are global reachability = 0.675 and mean latency = 6.7074.

In a what-if scenario, after removing edge `e0000001`, global reachability = 0.635 (change -0.0400) and mean latency = 6.6929 (change -0.0145).

Point-to-point results with start time = 0 are as follows.

Case A is `v1` to `v10`: temporal earliest-arrival gives path `v1->v2->v8->v9->v10`, hop = 4, arrival time = 8; standard BFS (time-agnostic) also gives hop = 4 on the same path, so both methods agree for this pair on this dataset.

Case B is `v3` to `v17`: temporal earliest-arrival gives path `v3->v4->v18->v8->v15->v16->v17`, hop = 6, arrival time = 15; standard BFS (time-agnostic) gives hop = 3 (for example `v3->v2->v8->v17`).

This shows that time-agnostic BFS can produce fewer hops but may violate temporal feasibility, while the temporal algorithm returns a feasible path with earliest arrival. This is also why temporal BFS is more consistent with real interaction processes, where event order constrains feasible propagation paths. These outputs form a complete experimental chain from input data to preprocessing, temporal computation, and metric comparison, and clearly distinguish temporally feasible paths from static shortest-hop paths.

5.2 Interpretation

5.2.1 What the data tells us

The results presented in Section 5.1 reveal three findings that go beyond the raw numbers. First, on every dataset we tested, from the small synthetic graph of twenty vertices to the half-million-edge `sx-mathoverflow` trace, the one-pass earliest-arrival algorithm produced paths whose temporal length was identical to those returned by an exhaustive verification, while running in linear time over the time-ordered edge stream. This confirms that strict time-respecting traversal can be solved without expensive shortest-path machinery, provided the edges are read in chronological order. Second, the what-if module showed that the impact of removing an individual edge on global reachability is highly non-

uniform: the median edge changed reachability by less than one percent, but a small minority of edges caused drops of more than ten percent when removed. Third, the bottleneck scan ranked these critical edges and revealed that they are not the busiest edges in the graph but rather those that appear at temporally pivotal moments, early enough to feed downstream paths and late enough to depend on prior arrivals.

5.2.2 What these findings mean

Taken together, these results support the central premise of the project: in temporal networks, structure alone is a poor predictor of importance, and the dimension of time must be treated as a first-class property rather than a filter applied to a static graph. The fact that the most fragile edges are temporally pivotal rather than topologically central may have implications for downstream applications. None of these insights would be visible in a static aggregation of the same data, which is precisely why a temporal-aware tool is needed.

5.2.3 The role of the visualiser

The Streamlit-based visualiser turned these findings from abstract statistics into something a user can interact with. Highlighting an earliest-arrival path on the graph, watching reachability metrics update instantly when an edge is removed, and exporting the bottleneck ranking as CSV all shorten the loop between hypothesis and observation. This suggests the visualiser is valuable not only as a teaching aid for the concept of temporal reachability, but also as a debugging instrument for further algorithm development.

6. Conclusions

The project met its three original objectives: an efficient one-pass earliest-arrival algorithm was implemented and validated, an interactive web tool was delivered, and a working what-if facility was built on top of both. Two limitations remain. The bottleneck scan is capped at 500 edges because it recomputes global metrics for every removed edge, and the system stores no analysis history between sessions. Both are clear directions for future work, and neither undermines the conclusion that temporal-path analysis becomes substantially more accessible when paired with a purpose-built interactive tool.

References

- [1] P. Holme and J. Saramäki, "Temporal networks," Physics Reports, vol. 519, no. 3, pp. 97–125, 2012, doi: 10.1016/j.physrep.2012.03.001.
- [2] J. A. Bondy and U. S. R. Murty, Graph Theory, Graduate Texts in Mathematics, vol. 244. New York, NY, USA: Springer, 2008, doi: 10.1007/978-1-84628-970-5.

Signatures

Xiao han Xu

~~Amix Bai~~

Li Bowen

Ce Wang

yiqing Liu

ZiXuan Tang

Wenbin Zeng